

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Scott Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and on knowledge-intensive tasks, their performance falls behind task-specific architectures. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models which combine pre-trained parametric and non-parametric memory for language generation.

1. Introduction and Overview

Pre-trained neural language models learn a substantial amount of in-depth knowledge from data without any explicit supervision. While this development is exciting, such models have notable downsides: they cannot easily expand or revise their memory, they can produce 'hallucinations', and they struggle to access and precisely manipulate factual knowledge.

Hybrid models that combine parametric memory (a pre-trained seq2seq transformer) with non-parametric memory (a dense vector index of Wikipedia accessed via a neural retriever) can address these issues because knowledge can be directly revised and expanded, and accessed predictions can be inspected and traced back to specific source documents.

The primary contribution of this research is the RAG architecture, which endows pre-trained, parametric-only language models with a non-parametric memory using pre-trained Dense Passage Retrieval (DPR) to query a dense vector index of Wikipedia passages. We train the retriever and generator end-to-end for knowledge-intensive generation.

2. Methods: RAG-Sequence vs RAG-Token

We explore two formulations for combining parametric and non-parametric memory: RAG-Sequence and RAG-Token. Both models use an input sequence x to retrieve top- k documents z . When generating the target sequence y , they treat retrieved documents as latent variables marginalizing over them.

In RAG-Sequence, the model uses the same retrieved document to generate the complete sequence. Concretely, it treats the retrieved document as a single latent variable that is marginalized over via a top- k approximation to generate the output sequence. The probability $p(y|x)$ is calculated as the sum of $p(z|x) * p(y|x, z)$ across top documents.

In RAG-Token, the model can draw from different latent documents for each token. This allows the generator to synthesize content from several passages when producing an answer. The transition probability for each token y_t is computed by marginalizing over documents: sum of $p(z|x) * p(y_t | x, y_{1:t-1}, z)$.

3. Non-Parametric Memory and Dense Retrieval

The non-parametric memory component is built using Dense Passage Retrieval (DPR). DPR uses a dual-encoder architecture: a query encoder $q(x)$ and a passage encoder $d(z)$, both based on BERT. The retrieval score is the dot product of the query and passage representations: $\text{sim}(q(x), d(z)) = q(x)^T d(z)$.

We index 21 million 100-word passages from Wikipedia. We build an efficient Maximum Inner Product Search (MIPS) index using FAISS to retrieve top-k passages with sub-second latency. During fine-tuning, the document index is kept fixed while the query encoder and generator are updated jointly.

Evaluating on open-domain question answering benchmarks (Natural Questions, TriviaQA, WebQuestions), RAG models establish new state-of-the-art results, generating more factual, specific, and diverse responses than parametric-only seq2seq baselines.